

# *Configuración y Administración del Servidor FTP*

---

## *ACTIVIDAD 1*

*Francisco Javier Otero Herrero  
Grupo ATU | 17 de Junio de 2025*

# **Configuración y Administración del Servidor FTP**

# **Configuración y Administración del Servidor FTP**

***Vamos a imponer un sistema para un equipo de desarrollo de forma que no se pierdan las modificaciones que se realicen en el código y permita a varias personas trabajar de forma simultánea en un mismo proyecto.***

## **PREGUNTAS**

### **1. ¿Qué utilidades tiene un sistema controlador de versiones?**

Un sistema controlador de versiones (**VCS**, del inglés **Version Control System**, también conocido como **sistema de gestión de versiones o SCM, Software Configuration Management**) es una herramienta que gestiona los cambios en un conjunto de archivos o documentos a lo largo del tiempo. Sus utilidades principales son:

- ✓ **Historial Completo:** Mantiene un registro detallado de cada cambio realizado, quién lo hizo, cuándo y por qué. Esto permite rastrear la evolución del proyecto y revisar cualquier versión anterior en cualquier momento.
- ✓ **Colaboración Sincronizada:** Facilita que múltiples desarrolladores trabajen en el mismo proyecto simultáneamente sin sobrescribir el trabajo de los demás. Permite fusionar (*merge*) los cambios de diferentes personas de forma controlada.
- ✓ **Recuperación de Errores:** Permite revertir cambios, deshacer modificaciones problemáticas o restaurar versiones anteriores del código si se introduce un error o se rompe algo.
- ✓ **Ramificación (Branching) y Fusión (Merging):** Permite crear **ramas** aisladas del código para desarrollar nuevas características o corregir errores sin afectar la **versión principal** del proyecto (**la rama master o main**). Una vez terminadas, estas ramas pueden fusionarse de nuevo con la rama principal.
- ✓ **Resolución de Conflictos:** Ofrece mecanismos para identificar y ayudar a resolver conflictos que surgen cuando dos o más desarrolladores modifican la misma parte del código de manera incompatible.
- ✓ **Auditoría y Trazabilidad:** Proporciona un rastro de **auditoría** completo de todos los cambios, lo que es esencial para la revisión de código, el cumplimiento normativo y la comprensión de por qué se hicieron ciertas modificaciones.
- ✓ **Automatización de Despliegues:** Se integra fácilmente con herramientas de integración continua/despliegue continuo (**CI/CD**), automatizando la construcción y el despliegue de software a partir de versiones específicas del código.
- ✓ **Gestión de Lanzamientos (Releases):** Permite etiquetar versiones estables del software (ej. v1.0, v2.1), facilitando la gestión de lanzamientos y la reproducción de entornos de desarrollo y producción específicos.

# **Configuración y Administración del Servidor FTP**

## **2. ¿Qué tipo de sistemas de control de versiones existen? Exponer pros y contras.**

Existen principalmente dos tipos de sistemas de control de versiones, **Centralizados y Distribuidos**.

### ✓ **Sistemas de Control de Versiones Centralizados (CVCS - Centralized Version Control Systems):**

En este tipo de sistemas, hay un **único servidor central** que contiene todas las versiones del código del proyecto. Los desarrolladores se conectan a este servidor para obtener la última versión, realizar cambios y enviar sus modificaciones. Ejemplos: **CVS, Subversion (SVN), Perforce**.

#### ➤ **Pros:**

- **Simplicidad:** Más fáciles de configurar y administrar inicialmente, ya que solo hay un repositorio que mantener.
- **Control Centralizado:** Permite a los administradores tener un control granular sobre quién puede acceder y modificar qué partes del código.
- **Visibilidad Global:** Es fácil ver el estado actual del proyecto de todos los desarrolladores en un solo lugar.

#### ➤ **Contras:**

- **Punto Único de Fallo:** Si el servidor central falla o deja de estar disponible, nadie puede realizar check-ins, check-outs, ni acceder al historial. Se detiene el trabajo de todo el equipo.
- **Dependencia de la Red:** Se requiere una conexión de red constante al servidor central para realizar cualquier operación (*commits, actualizaciones, etc.*).
- **Riesgo de Pérdida de Datos:** Si el servidor central sufre una pérdida de datos irrecuperable y no hay copias de seguridad adecuadas, se puede perder todo el historial del proyecto.
- **Menos Flexibilidad en Ramificación:** Aunque soportan ramificación, suele ser más costosa y menos ágil que en los sistemas distribuidos.

# **Configuración y Administración del Servidor FTP**

## ✓ **Sistemas de Control de Versiones Distribuidos (DVCS - Distributed Version Control Systems)**

Cada desarrollador no solo obtiene la última versión del código, sino una copia completa del repositorio, incluyendo todo el historial. Esto significa que cada copia es un repositorio completo e independiente. Los desarrolladores pueden trabajar localmente y luego **push**, enviar cambios locales a un repositorio remoto, o **pull**, descarga cambios remotos al repositorio local. Ejemplos: **Git**, **Mercurial**, **Bazaar**.

### ➤ **Pros:**

- **Resiliencia y Redundancia:** No hay un punto único de fallo. Si el servidor principal (si lo hay) falla, los desarrolladores pueden seguir trabajando con sus repositorios locales y, una vez que el servidor se recupere, sincronizar los cambios, o incluso designar otro repositorio local como el **central**.
- **Velocidad:** Las operaciones más comunes (*commits*, *ver historial*, *ramificación*, *fusión*) se realizan localmente, lo que las hace mucho más rápidas y no dependen de la conexión de red.
- **Trabajo Offline:** Los desarrolladores pueden trabajar sin conexión a internet y sincronizar sus cambios más tarde.
- **Flexibilidad en Flujos de Trabajo:** Permiten una gran variedad de flujos de trabajo de colaboración (*centralizado*, *ramificación de características*, *integración continua*, etc.). La **ramificación** y **fusión** son operaciones rápidas y eficientes.
- **Colaboración Peer-to-Peer:** Los desarrolladores pueden compartir cambios directamente entre sí sin necesidad de un servidor central.

### ➤ **Contras:**

- **Mayor Curva de Aprendizaje:** Pueden ser más complejos de entender y usar inicialmente debido a la naturaleza distribuida y los conceptos adicionales (*ramas locales/remotas*, *staging area*, *rebase*, etc.).
- **Mayor Tamaño de Repositorios Locales:** Cada clonación descarga todo el historial, lo que puede resultar en repositorios locales grandes si el historial es muy extenso o los archivos son muy grandes.
- **Gestión de Binarios:** No están optimizados para el control de versiones de archivos binarios grandes y cambiantes (ej., *assets de juegos*, *archivos CAD*), aunque existen extensiones (**como Git LFS**) para mitigar esto.
- **Menos Control Centralizado Estricto:** Es más difícil para un administrador imponer políticas de acceso estrictas sobre quién puede hacer qué en ramas o archivos específicos antes de que los cambios lleguen al repositorio principal.

## **Configuración y Administración del Servidor FTP**

- 3. Realiza una búsqueda por internet e indica los principales programas (de control de versiones) para cubrir las necesidades del equipo de desarrollo expuestas en el enunciado.**

Para las necesidades de un equipo de desarrollo que busca no perder modificaciones y trabajar simultáneamente en un mismo proyecto, **los sistemas de control de versiones distribuidos (DVCS)** son, con mucha diferencia, la opción preferida y el estándar de la industria hoy en día. Los principales programas que cubrirían estas necesidades son:

### ➤ **Git:**

Es, sin duda, el sistema de control de versiones distribuido más popular y utilizado en el mundo. Desarrollado por Linus Torvalds (creador de Linux).

Excelente para la colaboración simultánea, ramificación y fusión eficientes (perfecto para características, corrección de errores sin afectar el tronco principal), rendimiento rápido (operaciones locales) y una comunidad enorme. La mayoría de las plataformas de hosting de código (GitHub, GitLab, Bitbucket, Azure DevOps) lo soportan nativamente.

Se puede utilizar para prácticamente cualquier proyecto de desarrollo de software, desde pequeño a muy grande, en cualquier lenguaje o tecnología.

### ➤ **Mercurial (Hg):**

Otro sistema de control de versiones distribuido, similar a Git en su filosofía, pero a menudo considerado un poco más sencillo de aprender para los principiantes debido a una sintaxis y un modelo de datos ligeramente más simples.

También permite el trabajo simultáneo y la protección contra la pérdida de modificaciones. Su curva de aprendizaje es a veces menos pronunciada que la de Git.

Es para aquellos equipos que buscan una alternativa a Git que sea percibida como más fácil, aunque su adopción es menor que la de Git.

## **Configuración y Administración del Servidor FTP**

Aunque Git es el líder indiscutible, por completar un poco más esta lista, se podría mencionar:

### ➤ **Apache Subversion (SVN):**

El **CVCS** más popular antes de la llegada de Git. Aún se utiliza en algunos entornos empresariales que prefieren un modelo centralizado estricto.

Facilita la gestión de permisos centralizada. Pero es peor para colaboración distribuida, ramificación y fusión más lentas, punto único de fallo.

No cumpliría tan eficientemente la necesidad de varias personas trabajando de forma simultánea con la agilidad que ofrecen los **DVCS**.

Para el equipo de desarrollo que se propone en esta actividad, **Git** sería la recomendación principal y casi universal. Proporciona la flexibilidad, la seguridad de las modificaciones, la capacidad de ramificación para el trabajo simultáneo sin interferencias y la resiliencia que necesitan. Plataformas como **GitHub**, **GitLab** o **Bitbucket** complementan perfectamente **Git**, ofreciendo una interfaz web, revisiones de código, etc.